

实验报告

实验题目：简单函数绘图语言词法分析器的设计与实现

一、概述

本实验利用 Python 语言和 re 模块设计并实现了一个基于正则表达式的简单函数绘图语言词法分析器。

- 主要功能:
 - 支持关键字 (plot, func, range)、标识符、整数、浮点数、运算符 (+, -, *, /, ^, =) 和分隔符 ((,), ,, ;, [,]) 的精准识别。
 - 支持跳过空白字符 (空格、制表符、换行符)。
 - 支持基础的非法字符错误定位 (例如非法字符 #, @)。

二、实验方案

1. 结构设计:
 - Token 类: 存储 type 和 value 的二元组。
 - Lexer 类: 封装源代码、位置指针、正则表达式模式及识别逻辑。
2. 正则式设计:

参考 实验一补充说明.md, 各 Token 的正则项按优先级由高至低排列:

 - KEYWORD: plot|range|func
 - FLOAT/INTEGER: 优先匹配浮点数 `\d+\.\d*` 后匹配整数 `\d+`。
 - OPERATOR: 使用字符集 `[+ \- */ ^ =]`。
 - IDENTIFIER: `[a-zA-Z][a-zA-Z0-9]*`。
3. 词法分析器的状态转换逻辑:
 - 从 Initial 状态出发, 按定义的 token_specs 进行正则表达式的前向匹配。
 - 若匹配成功, 若类型为 SKIP 则更新指针并重置扫描; 否则更新指针并生成 Token 对象返回。
 - 若匹配失败, 则抛出 SyntaxError。
4. 程序流程图逻辑:

输入代码 -> 指针 pos = 0 -> 正则全模式扫描 -> 判断匹配类型 (SKIP? / VALID_TOKEN? / MISMATCH?) -> 返回 Token -> 循环直至 EOF。

三、实验过程及结果分析

3.1 代码实现

```
import re

class Token:
    def __init__(self, type_, value):
        self.type = type_
        self.value = value

    def __repr__(self):
        return f"Token({self.type!r}, {self.value!r})"

class Lexer:
    def __init__(self, code):
        self.code = code
        self.pos = 0
        self.length = len(code)
        self.token_specs = [
            ('KEYWORD', r'plot|range|func'),
            ('FLOAT', r'\d+\.\d*|\.\d+'),
            ('INTEGER', r'\d+'),
            ('OPERATOR', r'[+ \- */ ^ =]'),
            ('PARENS', r'[(\)]'),
            ('BRACKETS', r'[\[\]]'),
            ('COMMA', r','),
            ('SEMICOLON', r';'),
            ('IDENTIFIER', r'[a-zA-Z][a-zA-Z0-9]*'),
            ('SKIP', r'\s+'),
            ('MISMATCH', r'.')
        ]
        self.pattern = '|'.join(f'(?P<{name}>{pattern})' for name, pattern in self.token_specs)

    def get_next_token(self):
        while self.pos < self.length:
            match = re.match(self.pattern, self.code[self.pos:])
            if not match: break
            type_ = match.lastgroup
```

```

        value = match.group()
        self.pos += len(value)
        if type_ == 'SKIP': continue
        if type_ == 'MISMATCH':
            raise SyntaxError(f"词法错误: 非法字符 '{value}' at index {self.pos-1}")
        return Token(type_, value)
    return Token('EOF', None)

```

3.2 测试过程 (全新测试用例)

测试案例 1: 复杂数学表达式

测试代码: $f(x) = \exp(x) * \tan(3.14 / 2);$

code = "f(x) = exp(x) * tan(3.14 / 2);"

分析: 该案例验证了科学函数作为标识符与常量的组合识别。

- 输出结果:

```

Token('IDENTIFIER', 'f')
Token('PARENS', '(')
Token('IDENTIFIER', 'x')
Token('PARENS', ')')
Token('OPERATOR', '=')
Token('IDENTIFIER', 'exp')
Token('PARENS', '(')
Token('IDENTIFIER', 'x')
Token('PARENS', ')')
Token('OPERATOR', '*')
Token('IDENTIFIER', 'tan')
Token('PARENS', '(')
Token('FLOAT', '3.14')
Token('OPERATOR', '/')
Token('INTEGER', '2')
Token('PARENS', ')')
Token('SEMICOLON', ';')

```

测试案例 2: 绘图指令与范围

测试代码: `plot g1(x) range [0, 100.];`

code = "plot g1(x) range [0, 100.];"

分析: 包含了带小数点的浮点数 100. 的识别及关键字 range。

- 输出结果:

```

Token('KEYWORD', 'plot')
Token('IDENTIFIER', 'g1')

```

```
Token('PARENS', '(')
Token('IDENTIFIER', 'x')
Token('PARENS', ')')
Token('KEYWORD', 'range')
Token('BRACKETS', '[')
Token('INTEGER', '0')
Token('COMMA', ',')
Token('FLOAT', '100.')
Token('BRACKETS', ']')
Token('SEMICOLON', ';')
```

测试案例 3：异常字符处理

测试代码: `y = x @ 1;`

```
code = "y = x @ 1;"
```

分析：检测 @ 符号触发报错。

- 输出结果：

词法错误：非法字符 '@' at index 6

四、总结与体会

1. **正则表达式的高效性**：通过一组定义良好的正则模式，词法分析器可以简洁地处理文本扫描，避免繁琐的多层 `if-else`。
2. **状态管理的重要性**：构建状态转换图有助于理清不同 `Token` 之间的语法边界，如区分 `.` 为浮点数起点还是其他符号。
3. **错误提示**：实验中发现提供位置索引有助于快速定位代码中的词法错误，优化了调试反馈。